

수) 소비자장석용추천예용텍체요충요한이

St $\frac{1}{2}$ C

S

S기대값은 $\frac{1}{2}$ C가 6

6 S?: ?C

$$\frac{t}{\sqrt{n}} \xrightarrow{\text{d}} N(0, 1) \quad \text{and} \quad \frac{5t}{\sqrt{n}} \xrightarrow{\text{d}} N(0, 5)$$

jj th

S

S i g
S
S
S 4
i g

3

4

S

i q

t 균형나 S

nt 74

jj d

2 S

노드(Node)

- ID: 고유 식별자
- 레이블(Label): 노드 타입
- 속성(Properties):
 - 일반 속성 (text, number, date 등)
 - 벡터 속성 (embedding)

- 타입(Type): 관계 종류
- 속성(Properties): 관계의 메타데이터

S

```
// 문서 노드에 벡터 저장
CREATE (doc:Document {
  id: 'doc123',
  title: 'AI in Healthcare',
```

```
content: '...',
embedding: [0.1, 0.2, ..., 0.n] // 1536차원 벡터
})

// 사용자 노드와의 관계
CREATE (user:User {name: 'Alice'})
CREATE (user)-[:INTERESTED_IN]->(doc)
```

nt가커 S

```
// 벡터 인덱스 생성
CREATE VECTOR INDEX doc_embedding_index
IF NOT EXISTS
FOR (n:Document)
ON n.embedding
OPTIONS {
  indexConfig: {
    `vector.dimensions`: 1536,
    `vector.similarity_function`: 'cosine',
    `vector.m`: 16,
    `vector.efConstruction`: 128,
    `vector.ef`: 40
  }
}
```

$$\frac{\text{평균 문서 길이} \times \text{평균 문서 길이}}{\text{평균 문서 길이} \times \text{평균 문서 길이}} = \frac{\text{평균 문서 길이} \times \text{평균 문서 길이}}{\text{평균 문서 길이} \times \text{평균 문서 길이}}$$

j j q

hc

```
S
CALL db.index.vector.queryNodes('doc_embedding_index', 10, [0.1, 0.2, ..., 0.n])
YIELD node, score
RETURN node.title, score
ORDER BY score DESC
```

$$\frac{\text{평균 문서 길이} \times \text{평균 문서 길이}}{\text{평균 문서 길이} \times \text{평균 문서 길이}} = \frac{\text{평균 문서 길이} \times \text{평균 문서 길이}}{\text{평균 문서 길이} \times \text{평균 문서 길이}}$$

i c

```
2 S
CALL db.index.vector.queryNodes('doc_embedding_index', 100, query_vector)
YIELD node, score
WHERE node.category = 'healthcare'
AND node.published_date > datetime('2024-01-01')
RETURN node.title, score
ORDER BY score DESC
LIMIT 10
```

j c

```
S
// 벡터 검색으로 초기 후보 찾기
CALL db.index.vector.queryNodes('doc_embedding_index', 50, query_vector)
YIELD node as similarDoc, score
```

```
// 그래프 탐색: 유사 문서와 관련된 다른 문서 찾기
MATCH (user:User {id: 'user123'})-[:VIEWED]->(doc:Document)
WHERE doc IN similarDoc
MATCH (doc)-[:RELATED_TO]->(relatedDoc:Document)

// 최종 점수: 벡터 유사도 + 그래프 관계 가중치
WITH relatedDoc, score,
     (score + 0.5) as hybridScore // 관계에 보너스 점수
ORDER BY hybridScore DESC
RETURN relatedDoc.title, hybridScore
```

nc

2 S

```
// 사용자와 유사한 사용자 찾기 (임베딩 기반)
CALL db.index.vector.queryNodes('user_embedding_index', 10, user_vector)
YIELD node as similarUser

// 유사 사용자가 좋아한 아이템 찾기 (그래프 관계 기반)
MATCH (similarUser)-[:RATED {score: >=4}]->(item:Item)
WHERE NOT (originalUser)-[:RATED]->(item)

// 아이템과 유사한 다른 아이템 찾기 (벡터 기반)
WITH item
CALL db.index.vector.queryNodes('item_embedding_index', 5, item.embedding)
YIELD node as similarItem, score

// 최종 추천 점수 계산
RETURN similarItem,
     (1.0 + score) * (COALESCE(item.popularity, 0.5)) as recommendation_score
ORDER BY recommendation_score DESC
```

oc

S

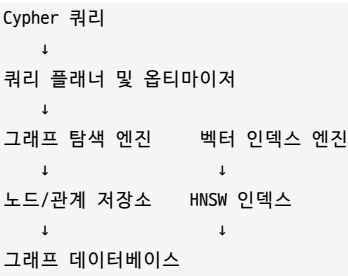
```
// 질문을 벡터로 변환
WITH vectorize('What are the latest treatments for cancer?') as question_vector

// 관련 문서 찾기
CALL db.index.vector.queryNodes('medical_doc_embedding_index', 20, question_vector)
YIELD node as document, score

// 관련 개념(노드) 탐색
MATCH (document)-[:MENTIONS]->(concept:Concept)
MATCH (concept)-[r:RELATED_TO]->(relatedConcept:Concept)

RETURN document.title, concept.name, relatedConcept.name, r.relationship_type
ORDER BY score DESC
LIMIT 10
```

j j m



S

Su/높은0nt가거
 S
 S
 S
 6 S
 Su/높은0/높 3가 0
 nt가거 Su/높은0/높a 0
 S
 S / 0
 S
 S 높답/ 0

jjϖ

S

S

S

S

jjϖ

j 딸타대단 5t 군나
 j 딸타대단 S
 =5 Sj 딸타대단 t 군나
 ?5 St 군나 j 딸타대단
 B5 St 군나
 C5 Sj 딸타대단

=5 2 S nt가거 d
 ?5 S 데답 d d
 B5 S d
 C5 S d
 D5 S d
 E5 S d